

Unit-VI

Planning

-Dr. Radhika V. Kulkarni

Associate Professor, Dept. of Computer Engineering,
Vishwakarma Institute of Technology, Pune.

Sources:

1. Stuart Russell & Peter Norvig, "Artificial Intelligence : A Modern Approach", Pearson Education, 2nd Edition.
2. Elaine Rich and Kevin Knight, "Artificial Intelligence" Tata McGraw Hill
3. Deepak Khemani, "A First Course in Artificial Intelligence", McGraw Hill Deepak Khemani, "A First Course in Artificial Intelligence", McGraw Hill

DISCLAIMER

This presentation is created as a reference material for the students of TY-AI, VIT (AY 2023-24).

It is restricted only for the internal use and any circulation is strictly prohibited.

Syllabus

Unit-VI : Planning

Blocks world, STRIPS, Implementation using goal stack, Planning with state space search: Forward state space search, Backward state space search, Heuristics for state space search. Partial Order Planning, Planning Graphs, Hierarchical planning, Least commitment strategy. Conditional Planning, Continuous Planning

Problem Solving: Problems & Solutions

- **Problems in Problem Solving:**
 - The problem-solving agent can be overwhelmed by irrelevant action
 - Finding a good heuristic function
 - Problem solver might be inefficient because it cannot take advantage of problem decomposition.
 - Problems with search agent
 - Too many actions and too many states to consider
 - Heuristic function can only choose among states and can't eliminate actions from consideration; so which action should be taken?
 - Agent is forced to consider actions starting from the initial state.
- **Solutions to Problem Solving:**
 - Planner is free to add actions to the plan wherever they are needed
 - So, it can make obvious and important decisions first, others later
 - Most parts of the world are independent of each other (nearly decomposable) → so, we can solve it independently (Divide and Conquer)
 - Open up the representation of states, goals, actions
 - States and goals are represented by sets of sentences; actions by logical descriptions of preconditions and effects -> *direct connections between states and actions*

Planning

- **Planning** is an important part of Artificial Intelligence which deals with the tasks and domains of a particular problem.
- **Planning is considered the logical side of acting. It is the task of coming up with a sequence of actions that will achieve a goal.**
- Everything we humans do is with a definite goal in mind, and all our actions are oriented towards achieving our goal. Similarly, Planning is also done for Artificial Intelligence.
- Through **planning** we see how an agent can take advantage of the structure of a problem to construct complex plans of action.
- Planning is about deciding the tasks to be performed by the artificial intelligence system and the system's functioning under domain-independent conditions.
- Problem-solving agents are able to plan ahead before acting. They are different in representing goals, states, and actions, and in ways of constructing action sequences.
- **A plan** is considered a sequence of actions, and each action has its preconditions that must be satisfied before it can act and some effects that can be positive or negative.
- **Execution of the plan** is about choosing a sequence of tasks with a high probability of accomplishing a specific task.

STRIPS Language (1)

- The Stanford Research Institute Problem Solver (STRIPS) is an automated planner developed by Richard Fikes and Nils Nilsson in 1971.
- STRIPS language is the base for most of the languages for expressing automated planning problem instances in use today; such languages are commonly known as action languages.
- A STRIPS instance is composed of:
 - An initial state;
 - The specification of the goal states – situations which the planner is trying to reach;
 - A set of actions. For each action, the following are included:
 - preconditions (what must be established before the action is performed);
 - postconditions (what is established after the action is performed).
- Mathematically, a STRIPS instance is a quadruple $\langle P, O, I, G \rangle$, in which each component has the following meaning:
 - P is a set of conditions (i.e., propositional variables);
 - O is a set of operators (i.e., actions); each operator is itself a quadruple $(\alpha, \beta, \gamma, \delta)$, each element being a set of conditions. These four sets specify, in order, which conditions must be true for the action to be executable, which ones must be false, which ones are made true by the action and which ones are made false;
 - I is the initial state, given as the set of conditions that are initially true (all others are assumed false);
 - G is the specification of the goal state; this is given as a pair $\langle N, M \rangle$, which specify which conditions are true and false, respectively, in order for a state to be considered a goal state.

STRIPS Language (2)

- A common language for writing STRIPS domain and problem sets is the [Planning Domain Definition Language \(PDDL\)](#).
- PDDL lets you write most of the code with English words, so that it can be clearly read and (hopefully) well understood. It's a relatively easy approach to writing simple AI planning problems.
- It describes the four things we need to define a search problem: the **initial state**, the **actions** that are available in a state, the **result** of applying an action, and the **goal test**.
- Each **state** is represented as a conjunction of fluents that are ground, functionless atoms.
- **Actions** are described by a set of action schemas that implicitly define the ACTIONS(s) and RESULT(s, a) functions needed to do a problem-solving search.
- A set of ground (variable-free) actions can be represented by a single **action schema**.
- The schema is a **lifted** representation—it lifts the level of reasoning from propositional logic to a restricted subset of first-order logic.
- The schema consists of the action name, a list of all the variables used in the schema, a **precondition** and an **effect**.
- For example, here is an action schema for flying a plane from one location to another:

Action(Fly(p, from, to),

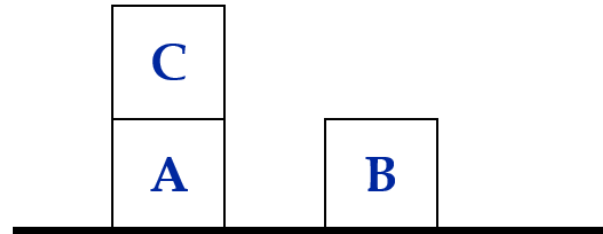
PRECOND: At(p, from) $\wedge$ Plane(p) $\wedge$ Airport (from) $\wedge$ Airport (to)

EFFECT: $\neg$ At(p, from) $\wedge$ At(p, to))

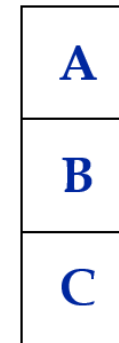
Blocks-World Planning Problem (1)

- One of the most famous planning domains is known as the [blocks-world](#).
- The [blocks-world problem](#) is known as the [Sussmann anomaly](#). The non-interlaced planners of the early 1970s were unable to solve this problem. Therefore, it is considered odd.
- It consists of a set of cube-shaped blocks sitting on a table. The blocks can be stacked, but only one block can fit directly on top of another. A robot arm can pick up a block and move it to another position, either on the table or on top of another block. The arm can pick up only one block at a time, so it cannot pick up a block that has another one on it. The goal will always be to build one or more stacks of blocks, specified in terms of what blocks are on top.
- The start position and target position are shown in the following diagram.

Initial:



Goal:



Blocks-World Planning Problem (2)

- We use *On(b, x)* to indicate that block b is on x, where x is either another block or the table.
- The action for moving block b from the top of x to the top of y will be *Move(b, x, y)*.
- Now, one of the preconditions on moving b is that no other block be on it. In first-order logic, this would be $\neg \exists x \text{ On}(x, b)$ or, alternatively, $\forall x \neg \text{On}(x, b)$. **Basic PDDL does not allow quantifiers**, so instead we introduce a predicate *Clear(x)* that is true when nothing is on x.
- The complete problem description is in Figure 10.3.

Init(*On*(A, Table) $\wedge$ *On*(B, Table) $\wedge$ *On*(C, A)
 $\wedge$ *Block*(A) $\wedge$ *Block*(B) $\wedge$ *Block*(C) $\wedge$ *Clear*(B) $\wedge$ *Clear*(C))
Goal(*On*(A, B) $\wedge$ *On*(B, C))
Action(*Move*(b, x, y),
 PRECOND: *On*(b, x) $\wedge$ *Clear*(b) $\wedge$ *Clear*(y) $\wedge$ *Block*(b) $\wedge$ *Block*(y) $\wedge$
 (b $\neq$ x) $\wedge$ (b $\neq$ y) $\wedge$ (x $\neq$ y),
 EFFECT: *On*(b, y) $\wedge$ *Clear*(x) $\wedge$ $\neg$ *On*(b, x) $\wedge$ $\neg$ *Clear*(y))
Action(*MoveToTable*(b, x),
 PRECOND: *On*(b, x) $\wedge$ *Clear*(b) $\wedge$ *Block*(b) $\wedge$ (b $\neq$ x),
 EFFECT: *On*(b, Table) $\wedge$ *Clear*(x) $\wedge$ $\neg$ *On*(b, x))

Figure 10.3 A planning problem in the blocks world: building a three-block tower. One solution is the sequence [*MoveToTable*(C, A), *Move*(B, Table, C), *Move*(A, Table, B)].

Blocks-World Planning Problem (3)

- The action *Move* moves a block *b* from *x* to *y* if both *b* and *y* are clear. After the move is made, *b* is still clear, but *y* is not. A first attempt at the *Move schema* is

Action(Move(b, x, y),

PRECOND: On(b, x) $\wedge$ Clear (b) $\wedge$ Clear (y),

EFFECT: On(b, y) $\wedge$ Clear (x) $\wedge$ $\neg$ On(b, x) $\wedge$ $\neg$ Clear (y)) .

- Unfortunately, this does not maintain *Clear* properly when *x* or *y* is the table. When *x* is the Table, this action has the effect *Clear(Table)*, but the table should not become clear; and when *y* =Table, it has then precondition *Clear(Table)*, but the table does not have to be clear for us to move a block onto it.
- To fix this, we do two things. First, we introduce another action to move a block *b* from *x* to the table:

Action(MoveToTable(b, x),

PRECOND:On(b, x) $\wedge$ Clear (b),

EFFECT:On(b,Table) $\wedge$ Clear (x) $\wedge$ $\neg$ On(b, x)) .

- Second, we take the interpretation of *Clear(x)* to be “there is a clear space on *x* to hold a block.” Under this interpretation, *Clear(Table)* will always be true. The only problem is that nothing prevents the planner from using *Move(b, x, Table)* instead of *MoveToTable(b, x)*.
- We could live with this problem—it will lead to a larger-than-necessary search space but will not lead to incorrect answers—or we could introduce the predicate *Block* and add *Block(b) $\wedge$ Block(y)* to the precondition of *Move*.

Goal Stack Planning

Sources:

1. Stuart Russell & Peter Norvig, "Artificial Intelligence : A Modern Approach", Pearson Education, 2nd Edition.
2. Elaine Rich and Kevin Knight, "Artificial Intelligence" Tata McGraw Hill
3. <https://apoorvdixit619.medium.com/goal-stack-planning-for-blocks-world-problem-41779d090f29>
4. <https://www.javatpoint.com/what-is-the-role-of-planning-in-artificial-intelligence>

Goal Stack Planning (1)

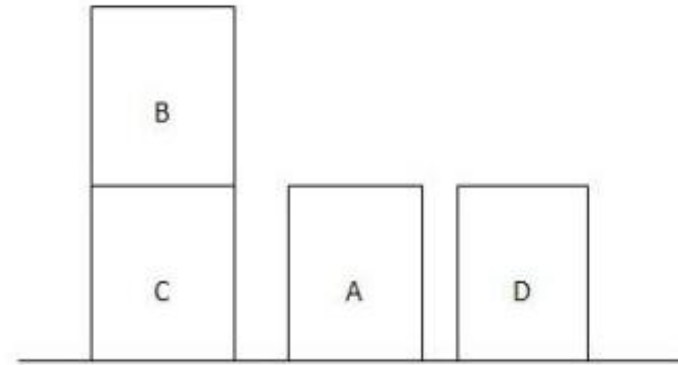
- **Goal stack planning** is a technique used in AI to create a set of goals that a computer system can pursue. The technique was first proposed by J. R. Searle in 1980.
- Its basic idea is to create a stack of goals, with each goal being pursued until it is achieved, at which point it is removed from the stack. It works backwards from the goal state to the initial state.
- The next goal in the stack is then pursued. This process continues until all goals in the stack have been achieved, at which point the system has solved the problem.
- Goal stack planning is a powerful technique that can be used to solve a wide variety of problems. It is particularly well suited to problems that can be decomposed into a set of sub-problems, each of which can be solved independently.
- **Advantages of goal stack planning:**
 - It is a complete planning algorithm. This means that, given a goal, the algorithm will always find a plan to achieve that goal, if one exists.
 - It is relatively efficient. In many cases, the algorithm will find a plan that is significantly shorter than the best possible plan.
- **Disadvantages of goal stack planning:**
 - It can be difficult to understand the plans that the algorithm produces.
 - The algorithm can sometimes produce very long plans. Finally, the algorithm can be used to solve only certain types of problems.

Goal Stack Planning (2)

- It is one of the most important planning algorithms used by STRIPS.
- Stacks are used in algorithms to capture the action and complete the target. A knowledge base is used to hold the current situation and actions.
- A goal/ target stack is similar to a node in a search tree, where branches are created with a choice of action.
- The important steps of the algorithm are mentioned below:
 1. Start by pushing the original target onto the stack. Repeat this until the pile is empty. If the stack top is a mixed target, push its unsatisfied sub-targets onto the stack.
 2. If the stack top is a single unsatisfied target, replace it with action and push the action precondition to the stack to satisfy the condition.
 3. If the stack top is an action, pop it off the stack, execute it and replace the knowledge base with the action's effect.
 4. If the stack top is a satisfactory target, pop it off the stack.

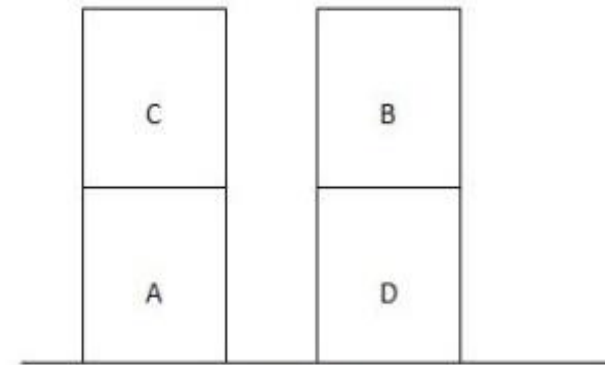
Goal Stack Planning (3)

- Example: Blocks-World



Initial State

$ON(B,C) \wedge ONTABLE(C) \wedge ONTABLE(A) \wedge ONTABLE(D)$
 $CLEAR(B) \wedge CLEAR(A) \wedge CLEAR(D)$



Goal State

$ON(C,A) \wedge ON(B,D) \wedge ONTABLE(A) \wedge ONTABLE(D)$
 $\wedge CLEAR(C) \wedge CLEAR(B)$

- The following list of operations can be performed by the robot arm in the various situation in our problem:
 - STACK(X,Y)** : Stacking Block X on Block Y
 - UNSTACK(X,Y)** : Picking up Block X which is on top of Block Y
 - PICKUP(X)** : Picking up Block X which is on top of the table
 - PUTDOWN(X)** : Put Block X on the table
- All the four operations have certain preconditions which need to be satisfied to perform the same. These preconditions are represented in the form of predicates.
- The effect of these operations is represented using two lists ADD and DELETE. **DELETE List** contains the predicates which will cease to be true once the operation is performed. **ADD List** on the other hand contains the predicates which will become true once the operation is performed.

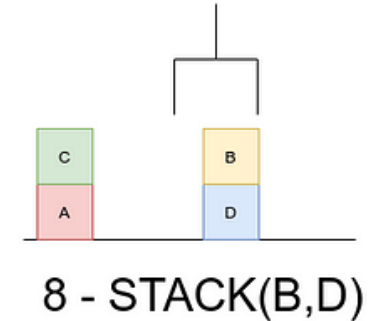
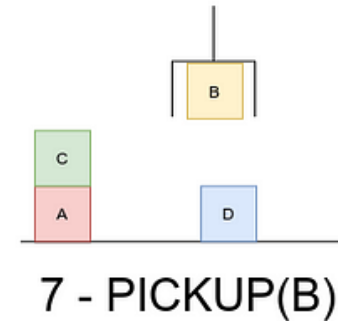
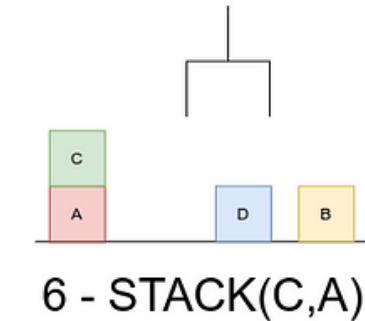
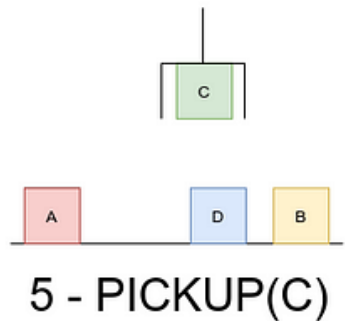
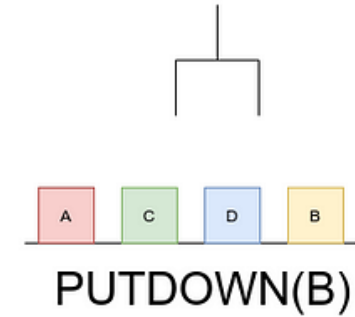
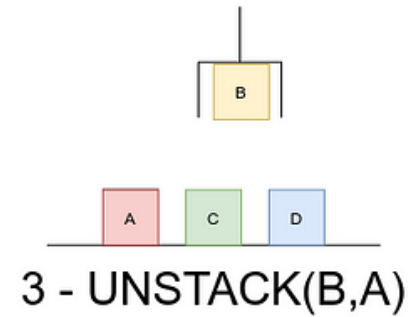
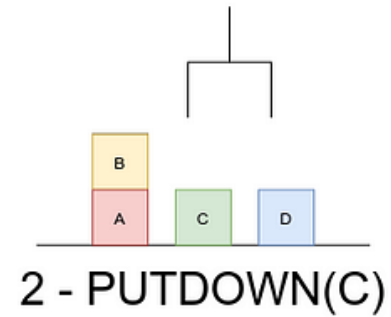
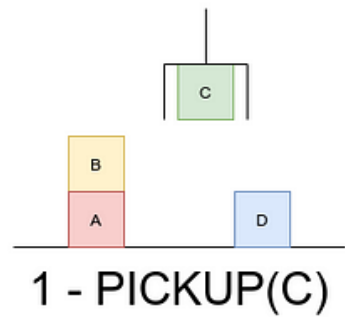
Goal Stack Planning (4)

- **Example: Blocks-World** (cont..)
- The Precondition, Add and Delete List for each operation is rather intuitive and have been listed below.

OPERATORS	PRECONDITION	DELETE	ADD
STACK(X,Y)	CLEAR(Y) $\wedge$ HOLDING(X)	CLEAR(Y) HOLDING(X)	ARMEMPTY ON(X,Y)
UNSTACK(X,Y)	ARMEMPTY $\wedge$ ON(X,Y) $\wedge$ CLEAR(X)	ARMEMPTY $\wedge$ ON(X,Y)	HOLDING(X) $\wedge$ CLEAR(Y)
PICKUP(X)	CLEAR(X) $\wedge$ ONTABLE(X) $\wedge$ ARMEMPTY	ONTABLE(X) $\wedge$ ARMEMPTY	HOLDING(X)
PUTDOWN(X)	HOLDING(X)	HOLDING(X)	ONTABLE(X) $\wedge$ ARMEMPTY

Goal Stack Planning (5)

- **Example: Blocks-World** (cont..)
- In this example, steps = [PICKUP(C), PUTDOWN(C), UNSTACK(B,A), PUTDOWN(B), PICKUP(C), STACK(C,A), PICKUP(B), STACK(B,D)]



Planning with State-Space Search

Sources:

1. Stuart Russell & Peter Norvig, "Artificial Intelligence : A Modern Approach", Pearson Education, 2nd Edition.
2. Elaine Rich and Kevin Knight, "Artificial Intelligence" Tata McGraw Hill
3. Deepak Khemani, "A First Course in Artificial Intelligence", McGraw Hill

Planning with State-Space Search (1)

- Basic Types of Planning in AI:

1. Forward State Space Planning (FSSP):

- It behaves in the same way as forwarding state-space search.
 - It says that given an initial state S in any domain, we perform some necessary actions and obtain a new state S' (which also contains some new terms), called a **progression**.
 - It continues until we reach the target position. Action should be taken in this matter.
-
- **Advantage:** The algorithm is Sound.
-
- **Disadvantages:**
 - It has large branching factor.
 - Forward search is prone to exploring irrelevant actions.
 - Forward state-space search was too inefficient to be practical.
 - It is hopeless without an accurate heuristic.

Planning with State-Space Search (2)

- Basic Types of Planning in AI: (cont..)

2. Backward State Space Planning (BSSP):

- It behaves similarly to backward state-space search.
- In this, we move from the target state **G** to the sub-goal **g**, tracing the previous action to achieve that goal. This process is called **regression** (going back to the previous goal or sub-goal).
- These sub-goals should also be checked for consistency.
- It is called **relevant-states search** because we only consider actions that are relevant to the goal (or current state).
- **Advantages:**
 - Small branching factor (much smaller than FSSP).
 - It explores only relevant actions.
- **Disadvantages:**
 - Backward search works only when we know how to regress from a state description to the predecessor state description.
 - Backward search uses state sets rather than individual states makes it harder to come up with good heuristics.
 - It is not sound algorithm (sometimes inconsistency can be found).

Planning with State-Space Search (3)

- Consider the task of flying a plane from one location to another.

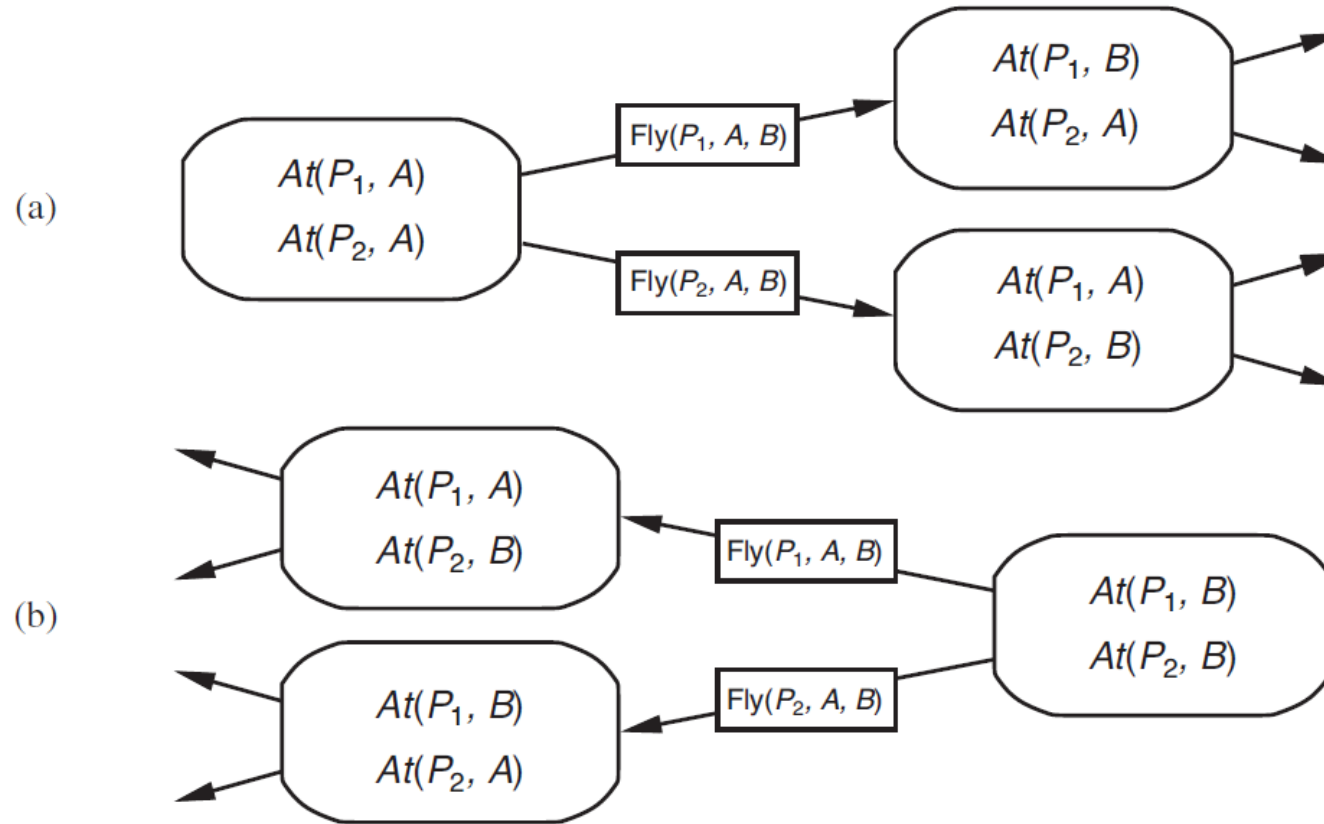


Figure 10.5 Two approaches to searching for a plan. (a) Forward (progression) search through the space of states, starting in the initial state and using the problem's actions to search forward for a member of the set of goal states. (b) Backward (regression) search through sets of relevant states, starting at the set of states representing the goal and using the inverse of the actions to search backward for the initial state.

Planning with State-Space Search (4)

- Basic Types of Planning in AI: (cont..)

3. Heuristics for planning:

- Neither forward nor backward search is efficient without a good heuristic function.
- Planning uses a factored representation for states and action schemas. That makes it possible to define good domain-independent heuristics and for programs to automatically apply a good domain-independent heuristic for a given problem.
- **Advantages:**
 - Heuristics makes search efficient.
- **Disadvantages:**
 - Selection of accurate heuristics is crucial task.

Planning with State-Space Search (4)

- Basic Types of Planning in AI: 3. Heuristics for planning:(cont..)
- **Selection of Heuristics:**
- Given a planning problem P,
 - Create a relaxed planning problem P' and use GraphPlan to solve it
 - Convert to set-theoretic representation
 - No negative literals; goal is now a set of atoms
 - Remove the delete lists from the actions
 - Construct a planning graph until a layer is found that contains all of the goal atoms
 - The graph will contain no mutexes because the delete lists were removed
 - Extract a plan π' from the planning graph
 - No mutexes $\rightarrow$ no backtracking $\rightarrow$ polynomial time
- $|\pi'|$ is a lower bound on the length of the best solution to P.

Partial-Order Planning

Sources:

1. Stuart Russell & Peter Norvig, "Artificial Intelligence : A Modern Approach", Pearson Education, 2nd Edition.
2. Elaine Rich and Kevin Knight, "Artificial Intelligence" Tata McGraw Hill
3. Deepak Khemani, "A First Course in Artificial Intelligence", McGraw Hill

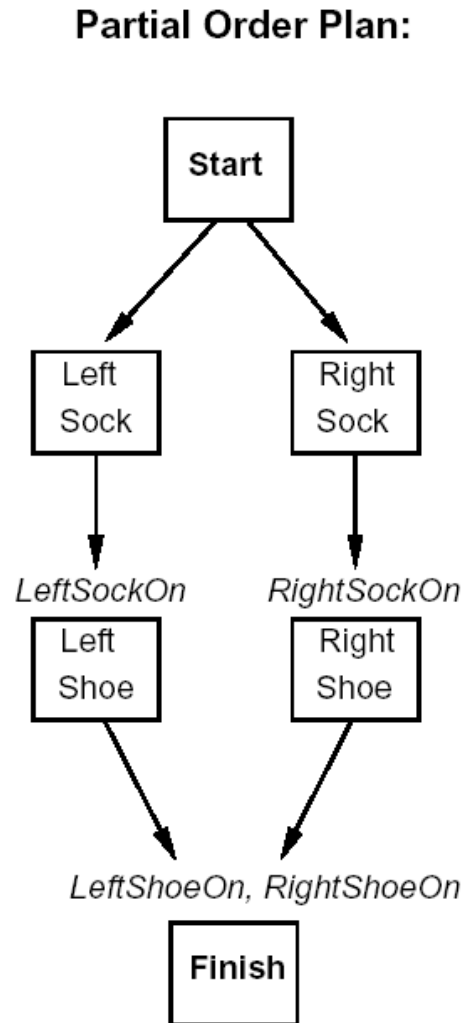
Partial-Order Planning (1)

- Partial-Order Planning (POP):
 - It is nondeterministic.
 - It starts with a minimal partial plan.
 - It satisfies one precondition at a time.
 - POP is a regression planner.
 - POP is sound and complete.
- Partial-order plan is a solution if:
 - All preconditions are supported (by causal links), i.e., no open conditions.
 - No threats
 - Consistent temporal ordering
- By construction, the POP algorithm reaches a solution plan.
- Heuristics for POP:
 - POP does not represent states directly, so it is harder to estimate how far a POP is from achieving a goal.
 - A key idea in defining heuristics is **decomposition**: dividing a problem into parts, solving each part independently, and then combining the parts. The subgoal independence assumption is that the cost of solving a conjunction of subgoals is approximated by the sum of the costs of solving each subgoal independently.
 - Heuristics are needed to choose which plan to refine.
 - Heuristic 1: to count the number of distinct open preconditions
 - Heuristic 2: the most-constrained-variable

Partial-Order Planning (2)

- Plan Generation: Search space of plans

- Nodes are partial plans
- Arcs/Transitions are plan refinements
- Solution is a node (not a path).
- Principle of “Least commitment”
 - e.g. do not commit to an order of actions until it is required



Total Order Plans:



Planning Graphs

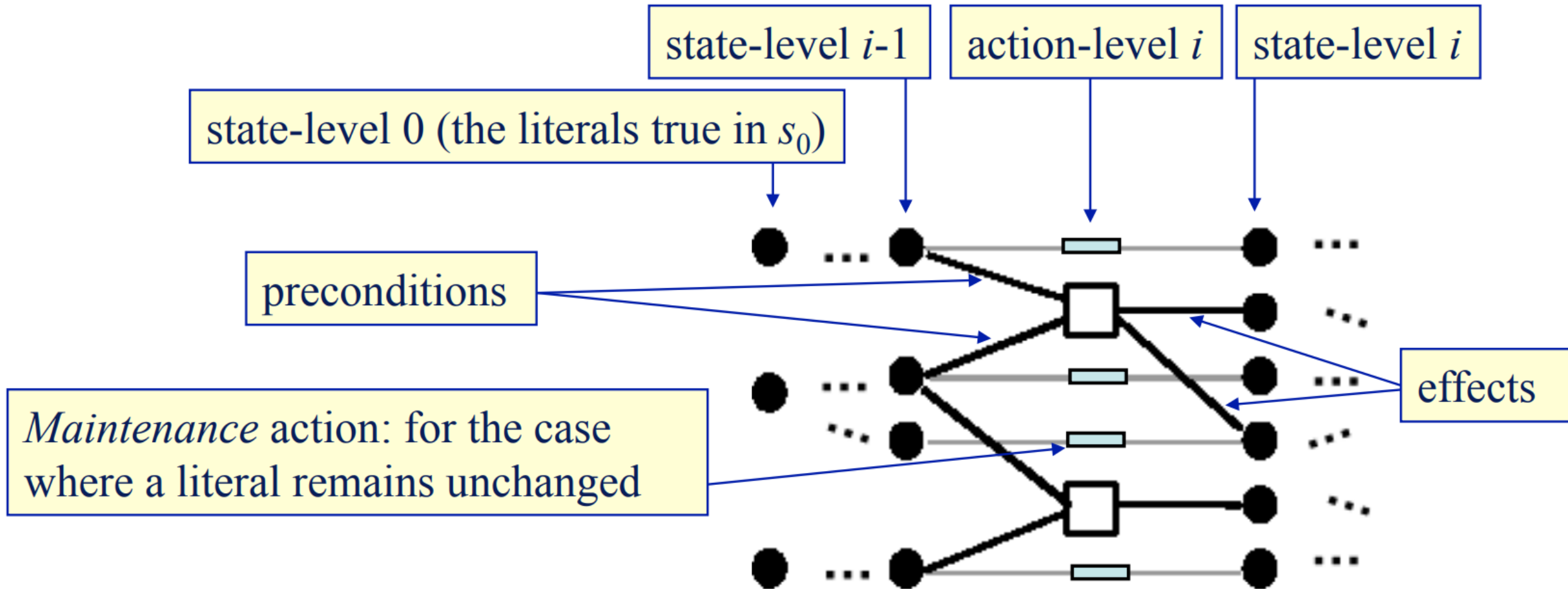
Sources:

1. Stuart Russell & Peter Norvig, "Artificial Intelligence : A Modern Approach", Pearson Education, 2nd Edition.
2. Elaine Rich and Kevin Knight, "Artificial Intelligence" Tata McGraw Hill
3. Deepak Khemani, "A First Course in Artificial Intelligence", McGraw Hill

Planning Graphs (1)

- A **planning graph** is a special data structure used to give better heuristic estimates. These heuristics can be applied to any of the search techniques.
- Alternatively, we can search for a solution over the space formed by the planning graph, using an algorithm called **GRAPHPLAN**.
- A planning graph is a directed graph organized into levels: first a level S_0 for the initial state, consisting of nodes representing each fluent that holds in S_0 ; then a level A_0 consisting of nodes for each ground action that might be applicable in S_0 ; then alternating levels S_i followed by A_i ; until we reach a termination condition.
- Roughly speaking, S_i contains all the literals that could hold at time i , depending on the actions executed at preceding time steps. If it is possible that either P or $\neg P$ could hold, then both will be represented in S_i . Also roughly speaking, A_i contains all the actions that could have their preconditions satisfied at time i .

Planning Graphs (2)



Planning Graphs (3)

$Init(Have(Cake))$
 $Goal(Have(Cake) \wedge Eaten(Cake))$
 $Action(Eat(Cake))$
 PRECOND: $Have(Cake)$
 EFFECT: $\neg Have(Cake) \wedge Eaten(Cake)$
 $Action(Bake(Cake))$
 PRECOND: $\neg Have(Cake)$
 EFFECT: $Have(Cake)$

Figure 10.7 The “have cake and eat cake too” problem.

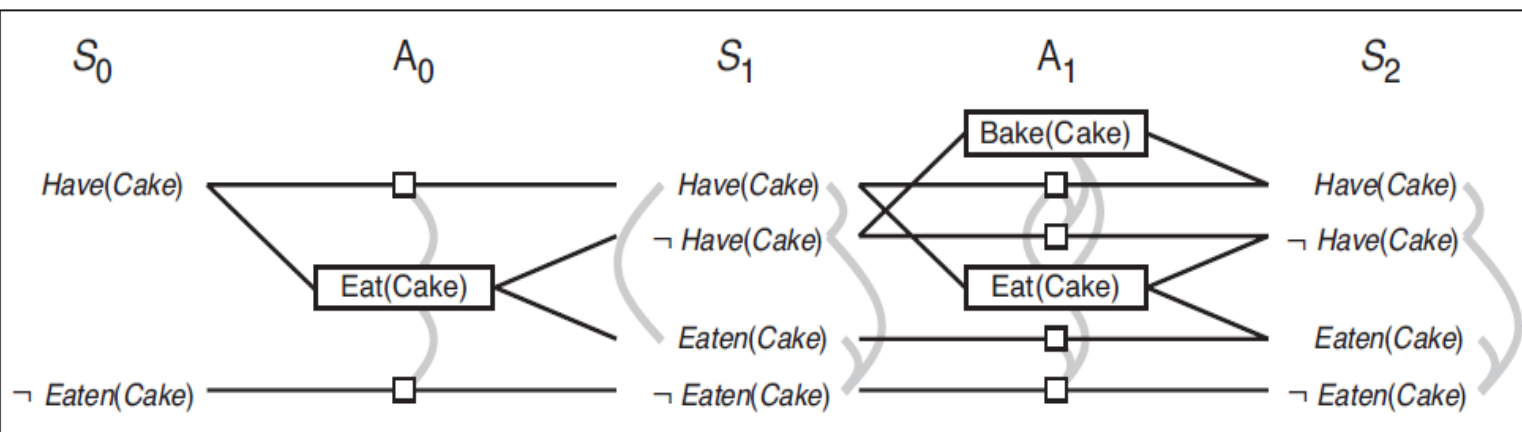


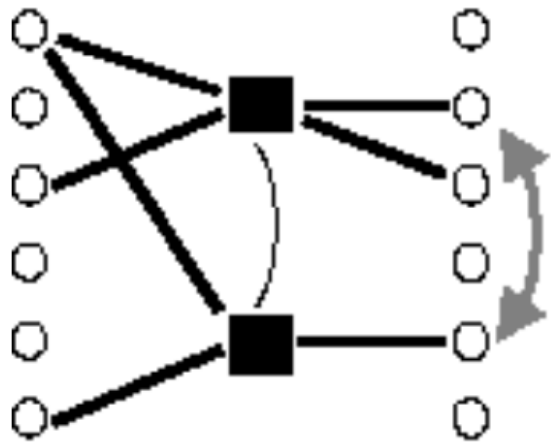
Figure 10.8 The planning graph for the “have cake and eat cake too” problem up to level S_2 . Rectangles indicate actions (small squares indicate persistence actions), and straight lines indicate preconditions and effects. Mutex links are shown as curved gray lines. Not all mutex links are shown, because the graph would be too cluttered. In general, if two literals are mutex at S_i , then the persistence actions for those literals will be mutex at A_i and we need not draw that mutex link.

Planning Graphs (4)

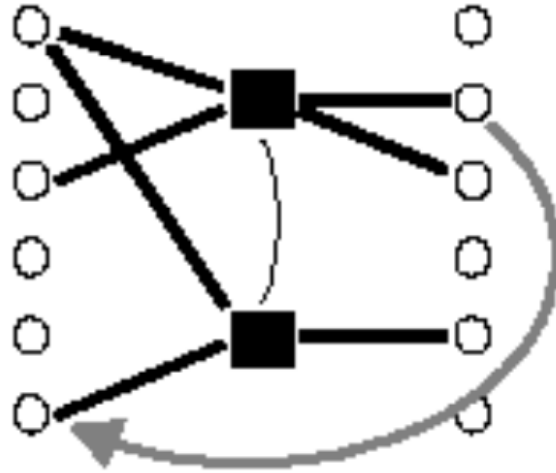
- **Mutual Exclusion:**
- **A mutex relation holds between two *actions* at a given level if any of the following three conditions holds:**
 - **Inconsistent effects:** an effect of one negates an effect of the other. For example, Eat(Cake) and the persistence of Have(Cake) have inconsistent effects because they disagree on the effect Have(Cake).
 - **Interference:** one deletes a precondition of the other. For example, Eat(Cake) interferes with the persistence of Have(Cake) by negating its precondition.
 - **Competing needs:** they have mutually exclusive preconditions. For example, Bake(Cake) and Eat(Cake) are mutex because they compete on the value of the Have(Cake) precondition.
- Otherwise, they don't interfere with each other
 - Both may appear in a solution plan
- **Two literals at the same state-level are mutex if**
- **Inconsistent support:** one is the negation of the other, or all ways of achieving them are pairwise mutex. For example, Have(Cake) and Eaten(Cake) are mutex in S_1 because the only way of achieving Have(Cake), the persistence action, is mutex with the only way of achieving Eaten(Cake), namely Eat(Cake). In S_2 the two literals are not mutex, because there are new ways of achieving them, such as Bake(Cake) and the persistence of Eaten(Cake), that are not mutex.

Planning Graphs (5)

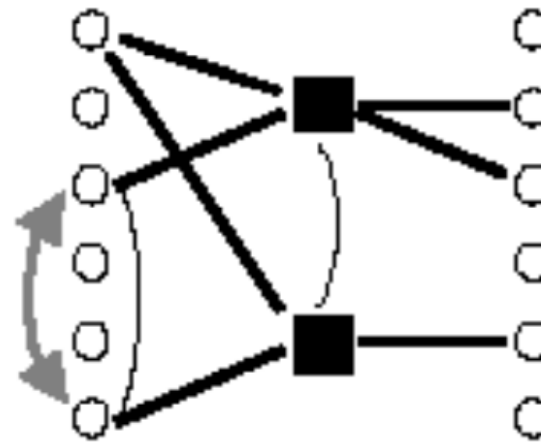
- **Mutual Exclusion:** (cont..)



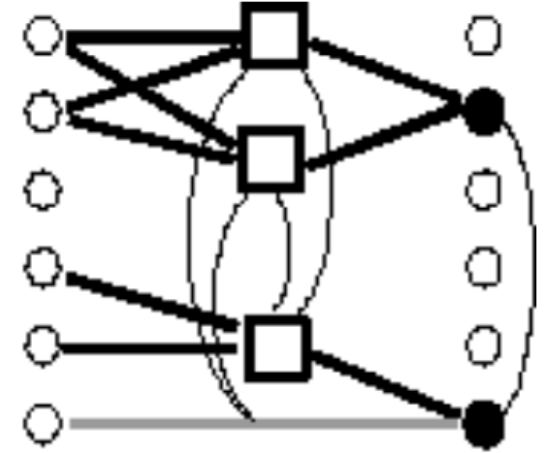
Inconsistent
Effects



Interference



Competing
Needs



Inconsistent
Support

Planning Graphs (6)

- **The GRAPHPLAN Algorithm:**
- The GRAPHPLAN algorithm (Figure 10.9) repeatedly adds a level to a planning graph with EXPAND-GRAPH. Once all the goals show up as nonmutex in the graph, GRAPHPLAN calls EXTRACT-SOLUTION to search for a plan that solves the problem. If that fails, it expands another level and tries again, terminating with failure when there is no reason to go on.

```
function GRAPHPLAN(problem) returns solution or failure  
graph ← INITIAL-PLANNING-GRAPH(problem)  
goals ← CONJUNCTS(problem.GOAL)  
nogoods ← an empty hash table  
for tl = 0 to  $\infty$  do  
    if goals all non-mutex in  $S_t$  of graph then  
        solution ← EXTRACT-SOLUTION(graph, goals, NUMLEVELS(graph), nogoods)  
        if solution  $\neq$  failure then return solution  
    if graph and nogoods have both leveled off then return failure  
    graph ← EXPAND-GRAPH(graph, problem)
```

Figure 10.9 The GRAPHPLAN algorithm. GRAPHPLAN calls EXPAND-GRAPH to add a level until either a solution is found by EXTRACT-SOLUTION, or no solution is possible.

Hierarchical Planning

Sources:

1. Stuart Russell & Peter Norvig, "Artificial Intelligence : A Modern Approach", Pearson Education, 2nd Edition.
2. Elaine Rich and Kevin Knight, "Artificial Intelligence" Tata McGraw Hill
3. Deepak Khemani, "A First Course in Artificial Intelligence", McGraw Hill

Hierarchical Planning (1)

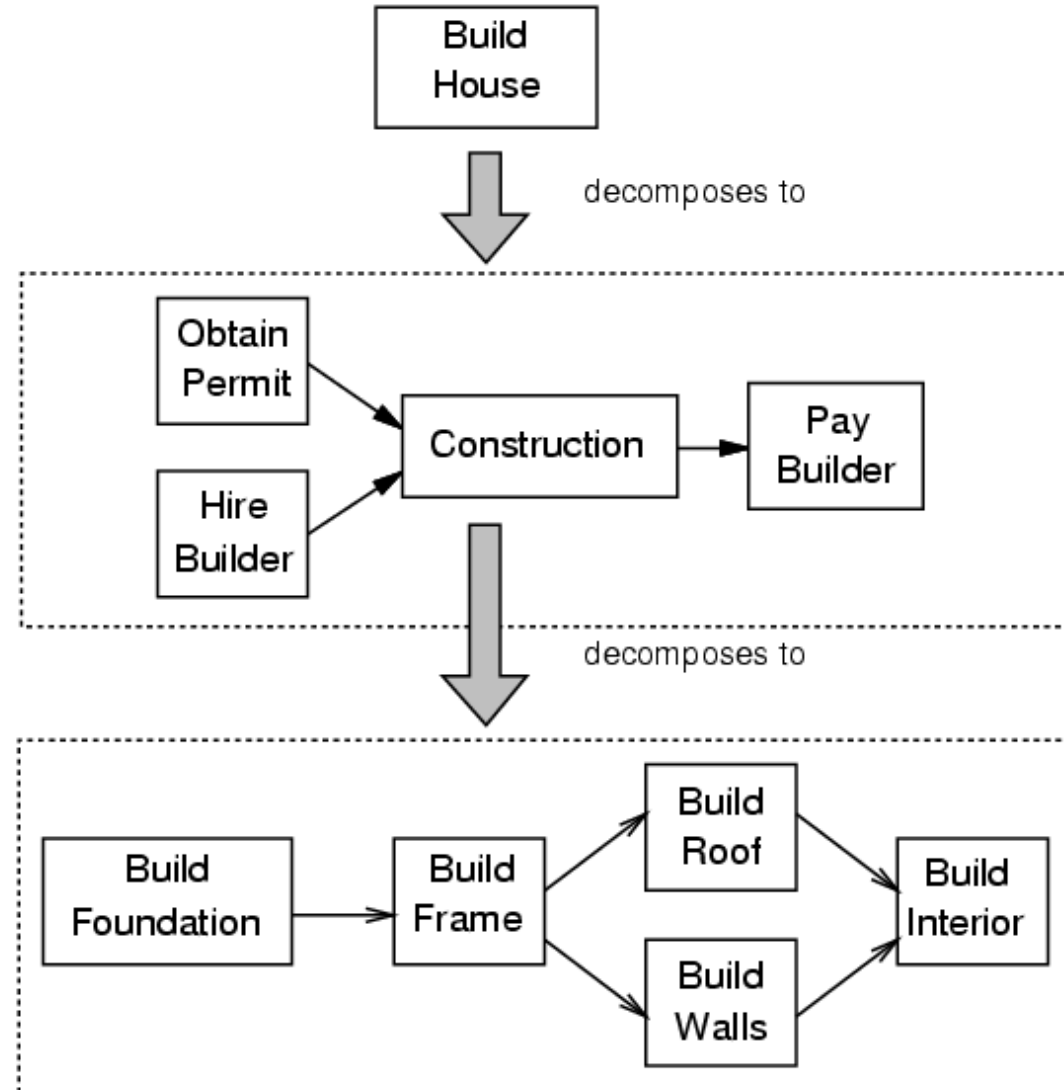
- Capture hierarchical structure of the planning domain
- Planning domain contains non-primitive actions and schemas for reducing them
- Reduction schemas:
 - given by the designer
 - express preferred ways to accomplish a task
- Hierarchical Problem reduction:
 - Decompose tasks into subtasks
 - Handle constraints
 - Resolve interactions
 - If necessary, backtrack and try other decompositions
- Basic Procedure:
 1. Input a planning problem P
 2. If P contains only primitive tasks, then resolve the conflicts and return the result. If the conflicts cannot be resolved, return failure.
 3. Choose a non-primitive task t in P
 4. Choose an expansion for t
 5. Replace t with the expansion
 6. Find interactions among tasks in P and suggest ways to handle them. Choose one.
 7. Go to 2

Hierarchical Planning (2)

- The basic formalism we adopt to understand hierarchical decomposition comes from the area of hierarchical task networks or HTN planning.
- **Hierarchical task network (HTN) planning** allows the agent to take advice from the domain designer in the form of high-level actions (HLAs) that can be implemented in various ways by lower-level action sequences.
- The effects of HLAs can be defined with angelic semantics, allowing provably correct high-level plans to be derived without consideration of lower-level implementations.
- HTN methods can create the very large plans required by many real-world applications.
- The key benefit of hierarchical structure is that, at each level of the hierarchy, a computational task, military mission, or administrative function is reduced to a small number of activities at the next lower level, so the computational cost of finding the correct way to arrange those activities for the current problem is small.
- Nonhierarchical methods, on the other hand, reduce a task to a large number of individual actions; for large-scale problems, this is completely impractical.

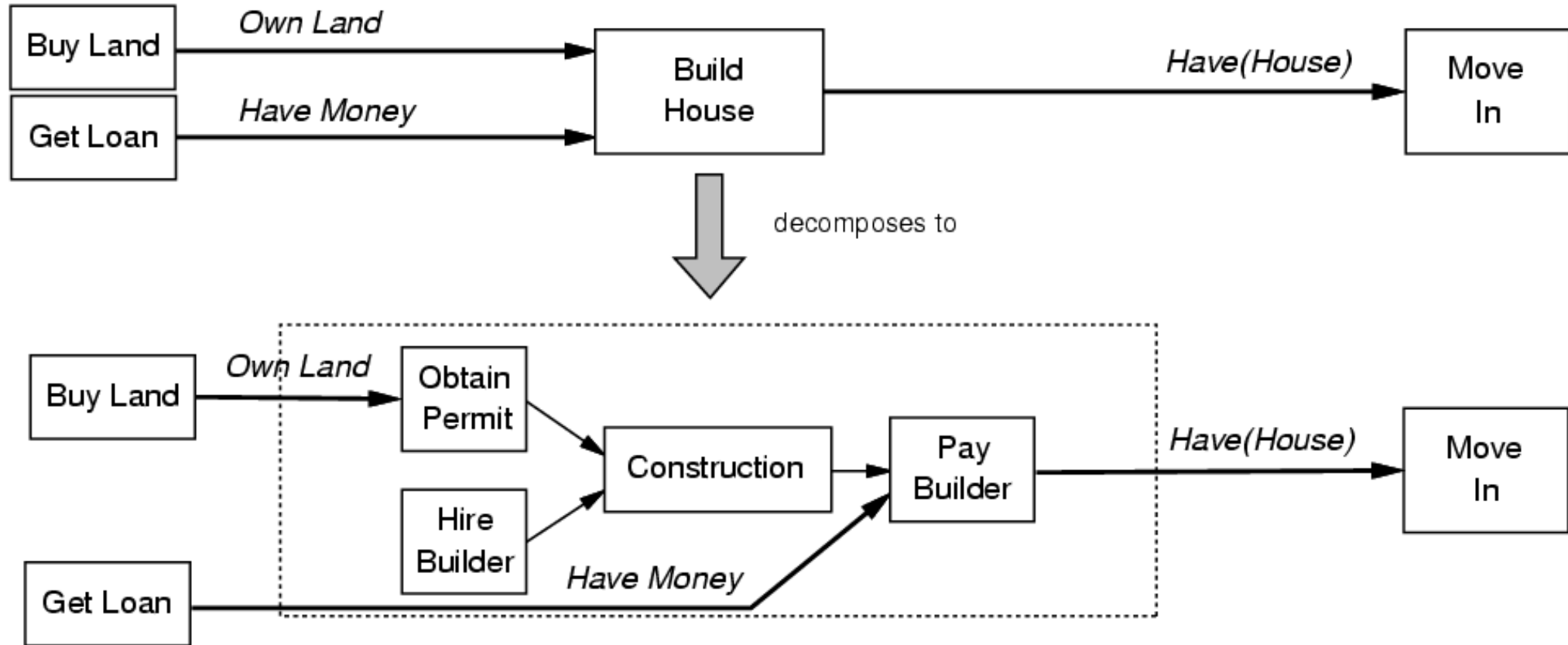
Hierarchical Planning (3)

- Hierarchical Decomposition:
- Example:



Hierarchical Planning (4)

- Task Reduction:
- Example: (cont..)



Planning & Execution

Initial State:



Move(x y)

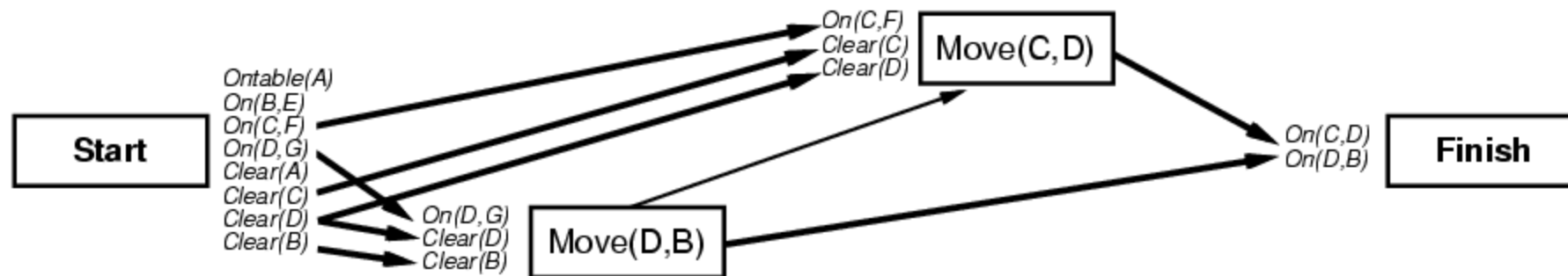
pre: $\text{clear}(x) \wedge \text{clear}(y) \wedge \text{on}(x z)$

eff: $\text{on}(x y) \wedge \text{clear}(z) \wedge$
 $\neg \text{on}(x z) \wedge \neg \text{clear}(y)$

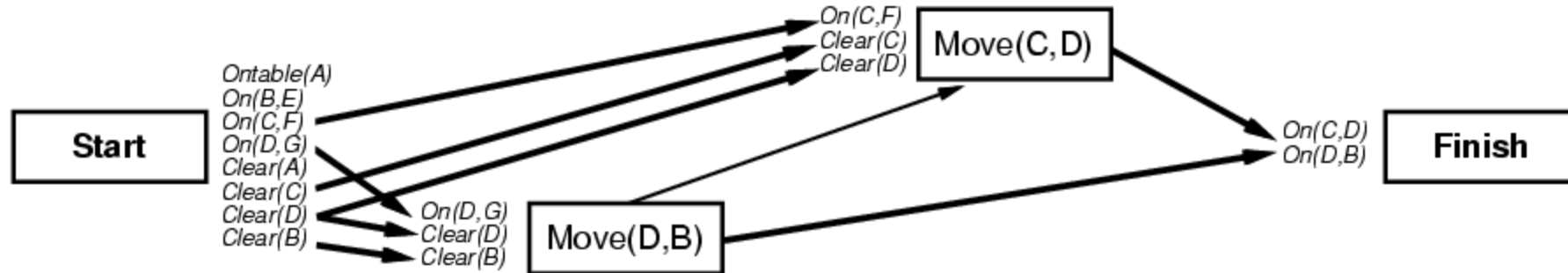
Goal:

On(C, D)

On(D, B)

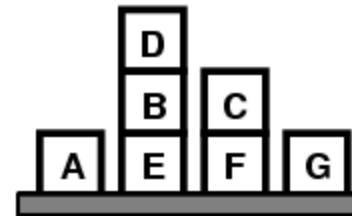


Plan ready to start execution

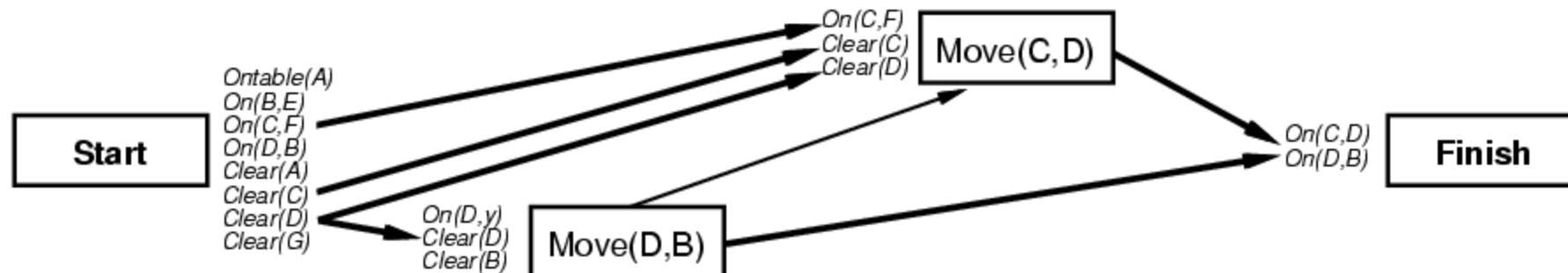


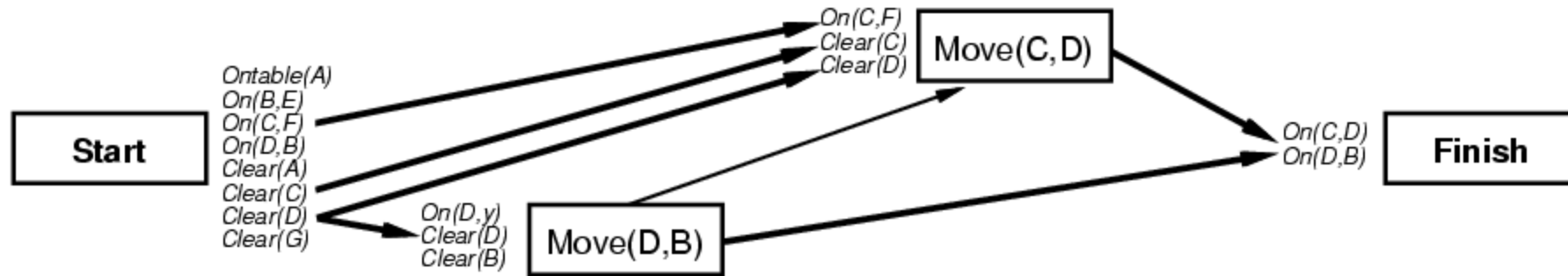
but genie intervenes: moves D to B !

New state of the world:



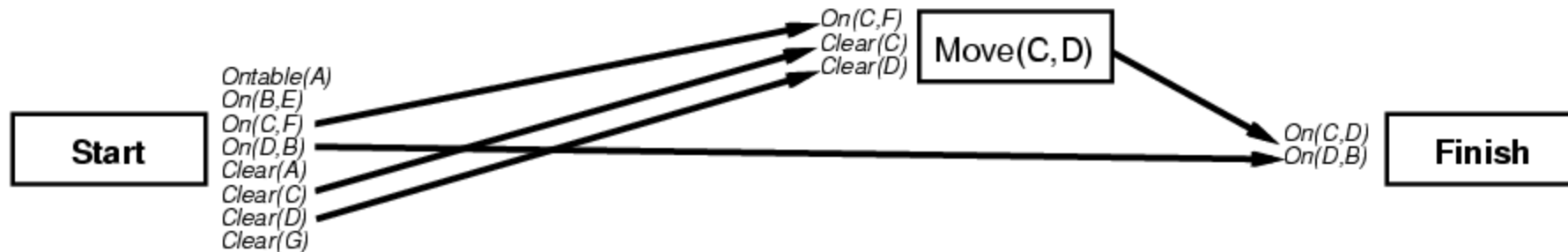
Updated plan:

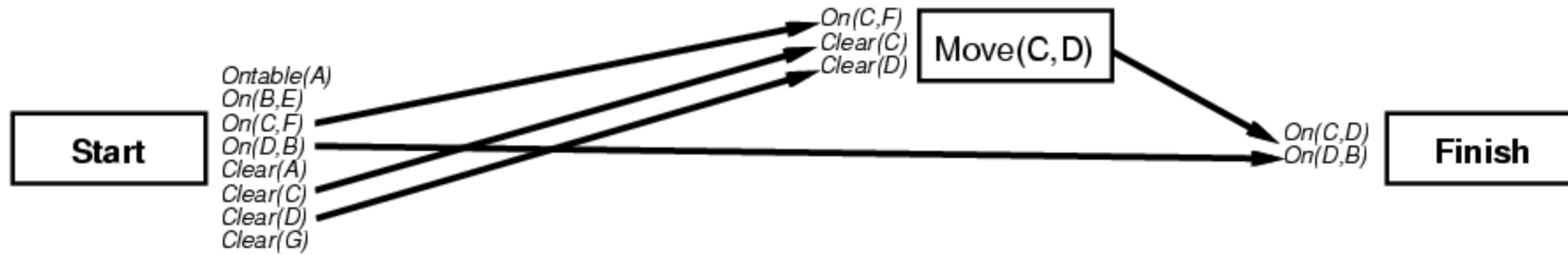




But it actually was a helpful interference:

- Can link to on(D B) from current state
- Move(D B) is now redundant

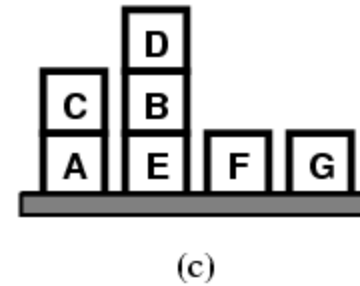




Now the agent can execute `move(C D)` to achieve the goal

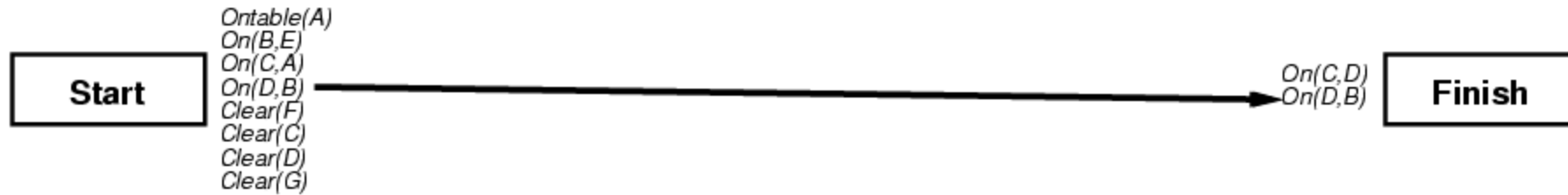
Unfortunately our agent is clumsy and drops C onto A instead of D

The new current state looks like:



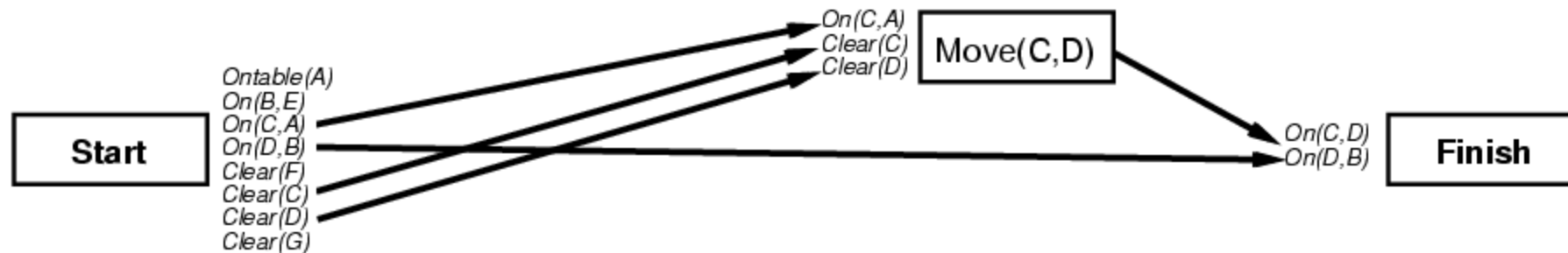
And the updated plan is:

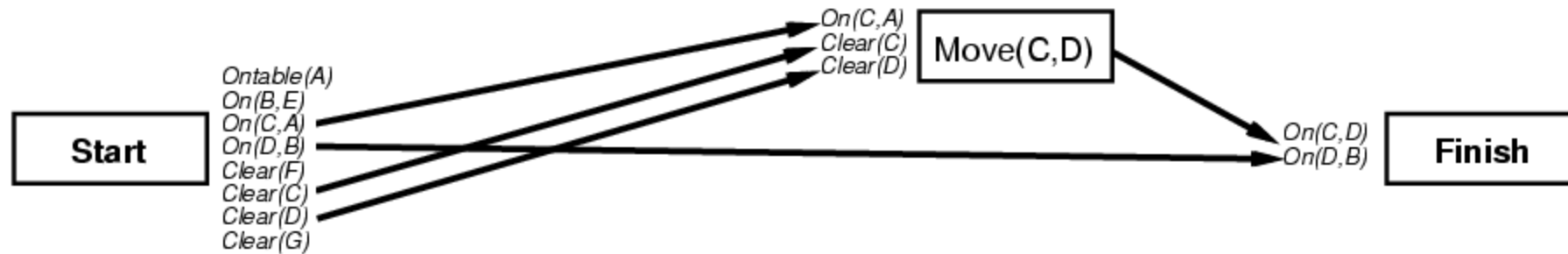




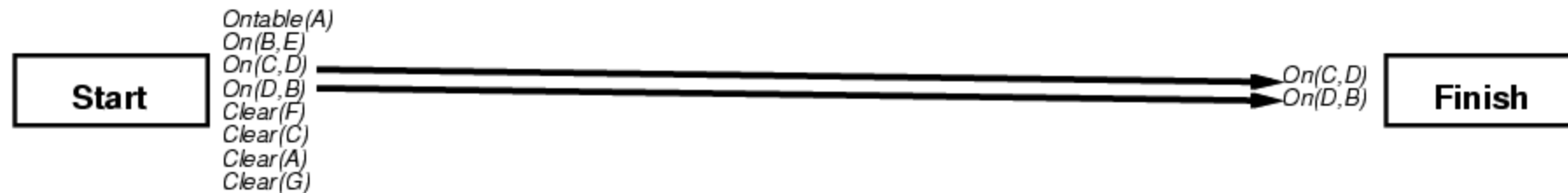
Keep planning to satisfy open condition on(C D)

Resulting plan:



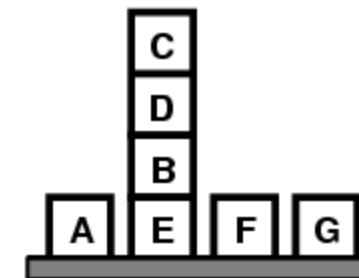


Fortunately, this time execution works:



The plan is finally completed:

- Goals achieved
- No threats
- No unexecuted step “flaws”



Conditional Planning

Sources:

1. Stuart Russell & Peter Norvig, "Artificial Intelligence : A Modern Approach", Pearson Education, 2nd Edition.
2. Elaine Rich and Kevin Knight, "Artificial Intelligence" Tata McGraw Hill
3. Deepak Khemani, "A First Course in Artificial Intelligence", McGraw Hill

Conditional Planning (1)

- It constructs a conditional plan with different branches for the different contingencies that could arise.
- It's a way to deal with uncertainty by checking what is actually happening in the environment at predetermined points in the plan. (Conditional Steps)
- Example: Check whether SFO airport is operational. If so, fly there; otherwise, fly to Oakland.
- There are **three kind of Environments**:
 1. **Fully Observable**: In this environment the agent always knows the current state.
 2. **Partially Observable**: In this environment the agent knows only a certain amount about the actual state. (much more common in real world).
 3. **Unknown**: In this environment the agent knows nothing about the current state.

Conditional Planning (2)

1. Conditional Planning in Fully Observable Environment:

- Agent used conditional steps to check the state of the environment to decide what to do next.
- Plan information stores in a library. E.g.: Action(Left) → Clean v Right
- Syntax: **If** <test> **then** plan_A **else** plan_B
- Use of AND-OR graphs

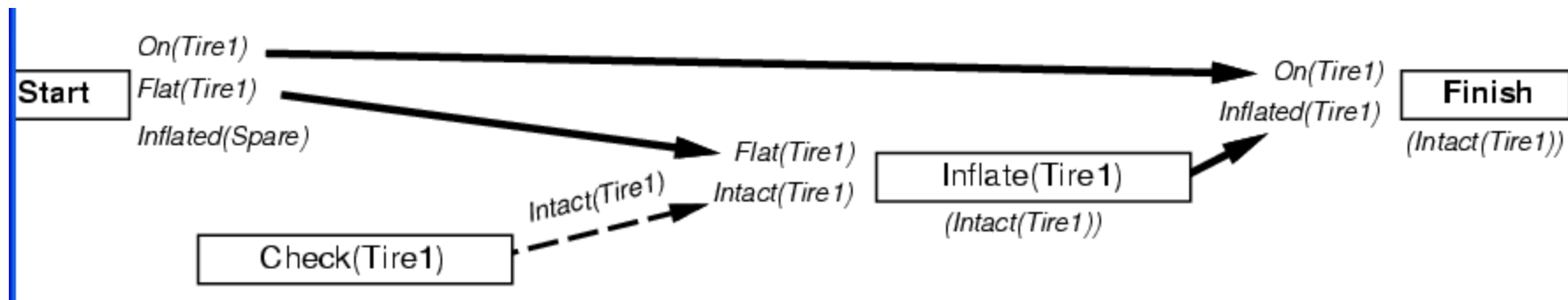
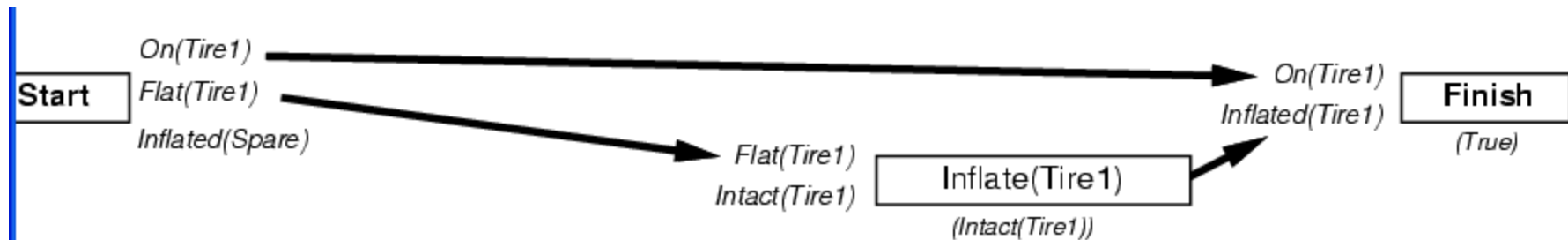
2. Conditional Planning in Partially Observable Environment:

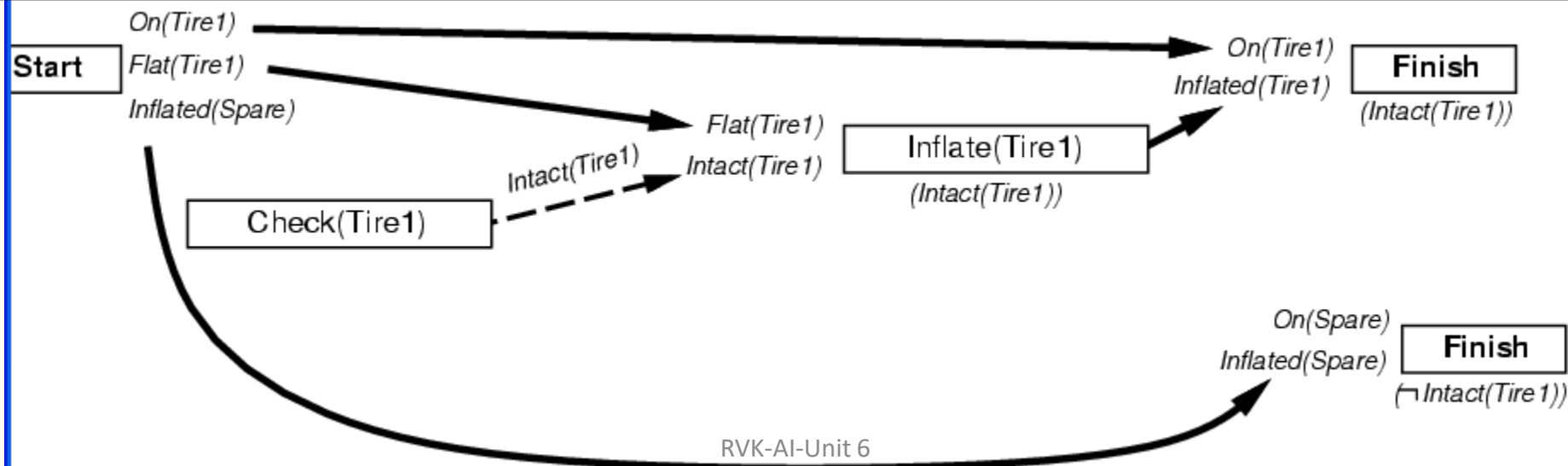
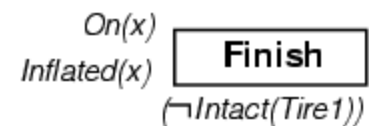
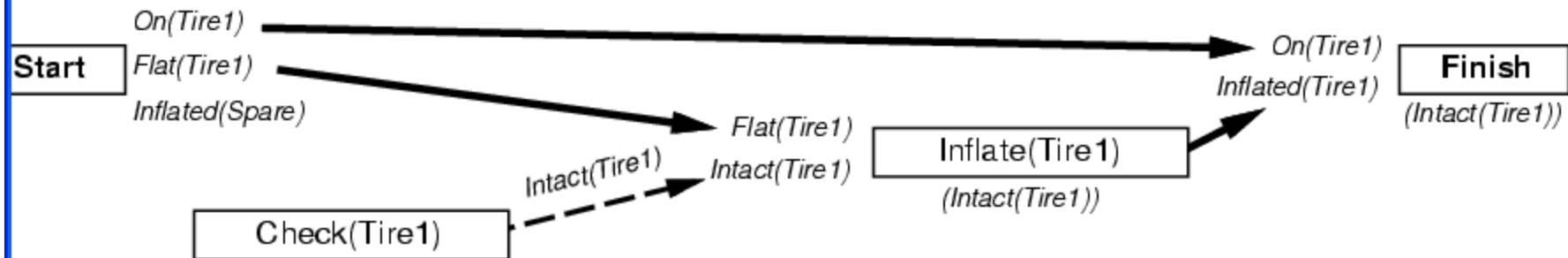
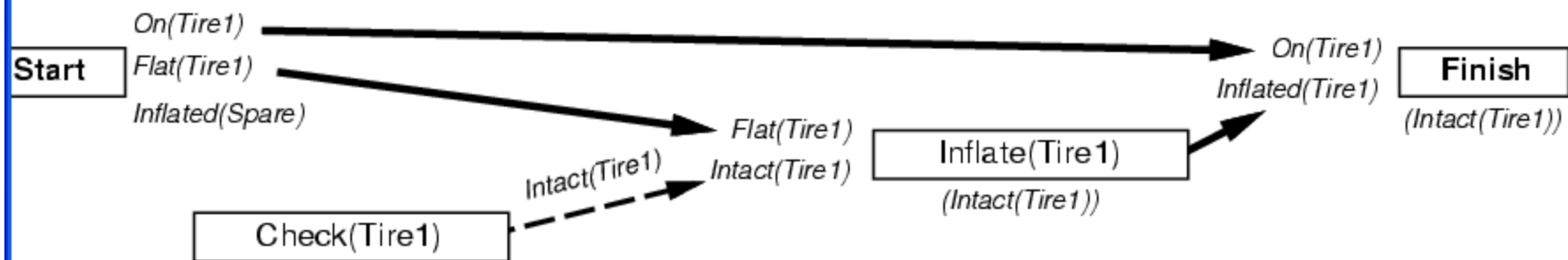
- It used the same AND-OR-Graph-Search algorithm, but the belief states will defly differently.
- Planning without observability by heuristic search.
- Three choices for belief states:
 - Sets of full state descriptions:** Ex: {(AtR and CleanR and CleanL), (AtR and CleanR and not CleanL)}. (not good, the size will become $O(2^n)$)
 - Logical sentences that capture exactly the set of possible worlds** (Open-world Assumption). Ex: AtR and CleanR. (not that much good, it can't represent all domains)
 - Knowledge Propositions** – describe the agent's knowledge (Closed-world Assumption). Ex: $K(P)$ means the agent knows that P is true, if it doesn't appear, it's assumed false.

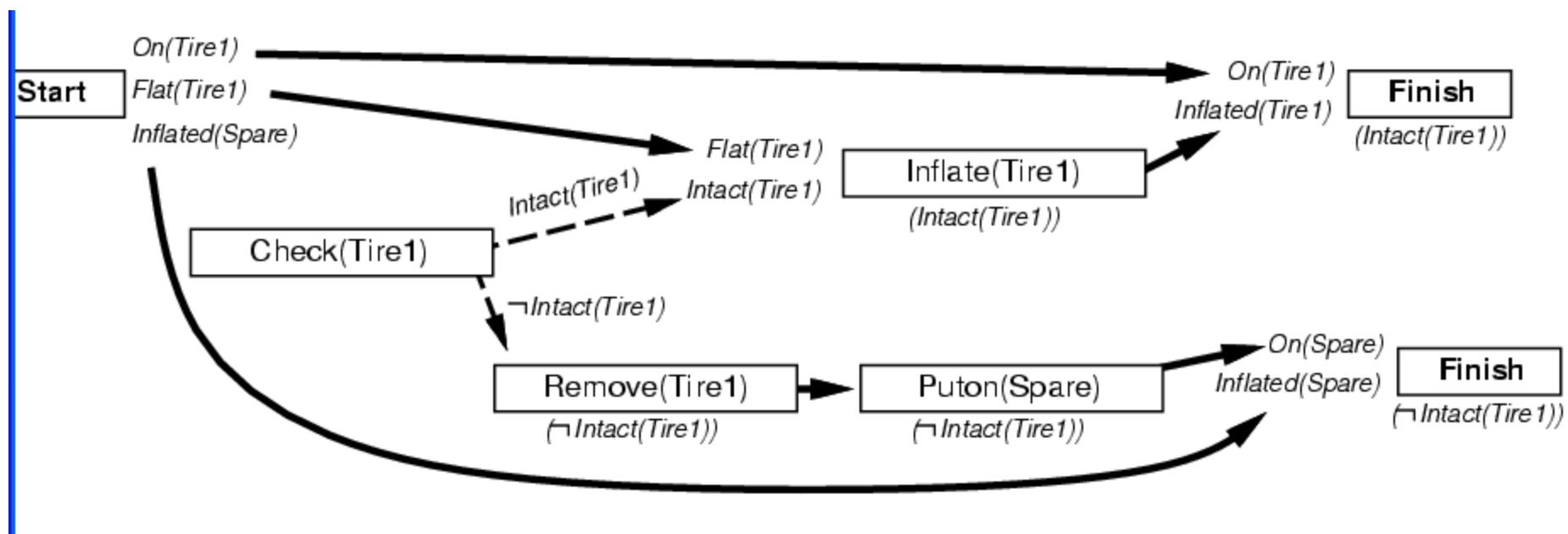
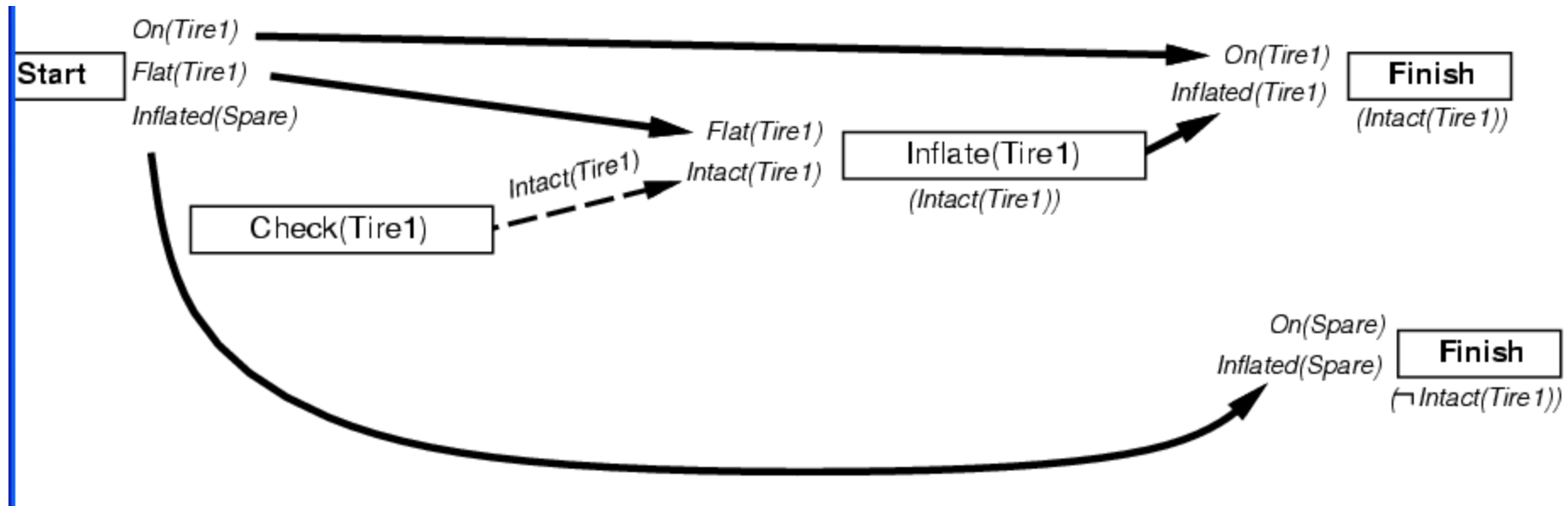
3. Conditional Planning in Unknown Environment: In this environment the agent knows nothing about the current state.

- Propositional logic is not suitable for representing planning with unobservability, and the language of quantified Boolean formulae is needed instead.
- Intuitively, the reason for this is that we have to quantify over an exponential number of plan executions

Conditional Planning







Thank you!